

Linux Administrator Command-Line Toolkit

20 PRACTICAL PAGES FOR JUNIOR ENGINEERS

- ✓ Essential Linux commands
- ✓ Real-world examples
- ✓ Step-by-step explanations
- ✓ Common mistakes and fixes
- ✓ Troubleshooting workflows
- ✓ Production best practices
- ✓ Build a strong admin foundation

PRACTICE
EXPLORE
TROUBLESHOOT
GROW



Learn
Commands



Solve
Real Problems



Build
Practical Skills



Be a
Confident
Linux Admin



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Know Where You Are: Navigation & Orientation

1. WHAT IS SHELL NAVIGATION?

- Navigation means moving around the file system using the command line.
- It helps you find files, check your current location and understand where you are before making changes.
- This is one of the most important skills for a Linux administrator.

2. IMPORTANT CONCEPTS

- **Absolute path:** Full path from root (/)
Example: `/home/user/projects`
- **Relative path:** Path from your current location
Example: `projects` (when inside `/home/user`)
- `.` means current directory
- `..` means parent directory (one level up)
- **Hidden files:** Files starting with a dot (`.`)

3. USEFUL COMMANDS

Command	What it does	Example
<code>pwd</code>	Shows your current directory path	<code>pwd</code>
<code>ls</code>	Lists files and directories	<code>ls</code>
<code>ls -lah</code>	Lists in long format, shows hidden files	<code>ls -lah</code>
<code>cd</code>	Changes directory	<code>cd /var/log</code>
<code>tree</code>	Shows directory structure (as a tree)	<code>tree</code>
<code>realpath</code>	Shows the absolute path of a file or directory	<code>realpath file.txt</code>

4. EXAMPLE OUTPUT

`$ pwd`

`/home/user`

`$ ls -lah`

```
total 24K
drwxr-xr-x  4 user user  4.0K Oct  1 10:00 .
drwxr-xr-x 20 user user  4.0K Sep 30 09:00 ..
-rw-r--r--  1 user user   220 Sep 30 09:30 .bashrc
drwxr-xr-x  2 user user  4.0K Oct  1 09:50 projects
-rw-r--r--  1 user user    0 Oct  1 09:55 file.txt
```

`$ tree`

```
.
├── file.txt
├── projects
│   ├── app
│   └── scripts
└── notes.txt
```

2 directories, 3 files

5. NAVIGATION EXAMPLE



6. COMMON TASKS

- Check where you are → `pwd`
- Go to a directory → `cd /path`
- Go up one level → `cd ..`
- List all files including hidden → `ls -lah`
- See full path of a file → `realpath file.txt`
- View directory structure → `tree`

7. BEGINNER MISTAKE



Running a command from the wrong directory.

You may think you are in the correct folder, but you are not. This can lead to editing, deleting or moving the wrong files, especially in production environments.

8. JUNIOR ENGINEER TIP



Always check your current location with `pwd` before running commands that modify, delete or move files. It helps you avoid costly mistakes.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Find Files Without Hunting Manually

1. WHY DO WE SEARCH FOR FILES?

- Linux servers can have thousands of files.
- You need to quickly find files, directories or executables without manually browsing.
- This saves time and helps you troubleshoot, check configuration and understand your system.

2. KEY CONCEPTS

- Find a file:** Search for files or directories anywhere in the filesystem (e.g., configuration files).
- Find an executable:** Find the command a shell will run (e.g., nginx, python).
- Different tools are used for different purposes.

3. USEFUL COMMANDS

Command	What it does	When to use it	Example
find	Searches for files or directories by name, type, size, date, etc.	Use when you need detailed search options and real-time results.	<code>find /etc -name "nginx.conf"</code>
locate	Searches a prebuilt database of files (faster).	Use for quick searches by name.	<code>locate nginx.conf</code>
which	Shows the full path of the executable a shell will run.	Use to find the location of a command in your PATH.	<code>which nginx</code>
whereis	Finds the binary, source and manual pages of a command.	Use to get more information about a command.	<code>whereis nginx</code>
type	Shows how the shell will interpret a command (alias, builtin or file).	Use to understand if a command is an alias, builtin or external executable.	<code>type nginx</code>

4. PRACTICAL EXAMPLES

```
# 1. Find a file by name
$ find / -name "nginx.conf" 2>/dev/null
/etc/nginx/nginx.conf
```

```
# 2. Find only files (not directories)
$ find /var -type f -name "*.log"
/var/log/syslog
/var/log/nginx/access.log
```

```
# 3. Find files larger than 100MB
$ find / -type f -size +100M 2>/dev/null
/var/lib/docker/large-file.img
```

```
# 4. Find files modified in the last 7 days
$ find /etc -type f -mtime -7
/etc/ssh/sshd_config
```

5. FIND COMMAND EXAMPLES

```
# Search by directory path
$ find /etc/nginx -name "*.conf"
/etc/nginx/nginx.conf
/etc/nginx/conf.d/app.conf
```

```
# Search by file type (directory)
$ find /var -type d -name "nginx"
/var/lib/nginx
```

6. LOCATE, WHICH, WHEREIS, TYPE EXAMPLES

```
# Using locate (faster, uses database)
$ locate nginx.conf
/etc/nginx/nginx.conf
/usr/share/doc/nginx/nginx.conf.gz
# Find the executable a shell will run
$ which nginx
/usr/sbin/nginx
# More information about a command
$ whereis nginx
nginx: /usr/sbin/nginx /usr/share/man/man8/nginx.8.gz
# How the shell interprets the command
$ type nginx
nginx is /usr/sbin/nginx
```

7. REAL-WORLD SCENARIO

Locate the SSH configuration file:

```
$ find /etc -name "sshd_config"
/etc/ssh/sshd_config
```

8. BEGINNER MISTAKE



Using locate and not finding a newly created file.
locate uses a database. If the file is new, it may not appear until the database is updated with updatedb.

9. JUNIOR ENGINEER TIP



Use find when you need accurate and flexible search options.
Use which and whereis when you want to know the exact location of a command.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Read Files & Inspect Configuration Safely

1. WHY THIS MATTERS

- As a Linux administrator, you will often read logs, configuration files and other data.
- You need to inspect files safely without accidentally modifying them.
- Some files are small and can be viewed directly, while others are very large.
- Using the right command makes you faster and prevents mistakes.

3. PRACTICAL EXAMPLES

```
# 1. View a small file with cat
$ cat /etc/hostname
veriqta-server
```

```
# 2. View a large file with less
$ less /var/log/syslog
```

Use arrow keys to navigate.
Press q to exit.

```
# 3. Show the first 10 lines
$ head /etc/nginx/nginx.conf
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
```

```
# 4. Show the last 10 lines
$ tail /var/log/nginx/access.log
192.168.1.10 - - [01/Oct/2026:10:20:01] ...
192.168.1.11 - - [01/Oct/2026:10:20:15] ...
192.168.1.12 - - [01/Oct/2026:10:20:30] ...
```

2. WHEN TO USE EACH COMMAND

Command	What it does	When to use it
cat	Shows the entire file content.	Small files only.
less	Views file page by page.	Large files (safe and recommended).
head	Shows the first lines of a file.	Quickly check the start of a file.
tail	Shows the last lines of a file.	Check the end of a file (e.g., logs).
tail -f	Continuously shows new lines as they are added.	Monitor a file in real time (e.g., application logs).
wc	Counts lines, words and bytes.	Get file size information.
file	Shows the file type and format.	Check what kind of file it is (text, binary, etc).

4. WATCH A LOG IN REAL TIME

```
# Follow a log file for new entries
$ tail -f /var/log/nginx/error.log
2026/10/01 10:21:15 [error] 1234#1234: open()
"/var/www/html/index.html" failed (2: No such file
or directory)
2026/10/01 10:21:20 [error] 1234#1234: client
192.168.1.10:54321 connected
```

Press Ctrl + C to stop.

5. GET FILE INFORMATION

```
# Count lines, words and bytes
$ wc /etc/nginx/nginx.conf
121  520  4820
lines words bytes
```

```
# Check the file type
$ file /etc/nginx/nginx.conf
/etc/nginx/nginx.conf:
ASCII text
```

6. BEGINNER MISTAKE



Using cat on a very large file.

This can fill your terminal screen, be hard to read and may freeze your session.
Use less for large files.

7. JUNIOR ENGINEER TIP



Use less as your default file viewer.

It allows you to search (press /), move through the file and exit easily. It is safer and works well for large logs and configuration files.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Search Inside Files & Logs

1. WHY USE GREP?

- Log files and configuration files can be very large.
- grep** helps you quickly find specific text or patterns instead of reading thousands of lines manually.
- This saves time and helps you identify errors, warnings and important information in seconds.

3. BASIC EXAMPLES

```
# 1. Search for a word in a file
$ grep "ERROR" /var/log/nginx/error.log
2026/10/01 10:21:15 [error] 1234#1234: open() failed
```

```
# 2. Case insensitive search
$ grep -i "failed" /var/log/nginx/error.log
2026/10/01 10:21:15 [error] open() FAILED
```

```
# 3. Show line numbers
$ grep -n "connection" /var/log/nginx/access.log
12:192.168.1.10 - - [01/Oct/2026:10:20:01] "GET ...
45:192.168.1.11 - - [01/Oct/2026:10:21:30] "POST ...
```

5. USE GREP WITH PATTERNS (grep -E)

```
# Find multiple keywords (error OR failed OR denied)
$ grep -E "error|failed|denied" /var/log/auth.log
Oct 01 10:21:15 server sshd[1234]: Failed password
Oct 01 10:22:10 server sudo: denied user root
Oct 01 10:25:11 server app[5678]: error connecting
```

The **|** symbol means **OR**. This finds lines containing error or failed or denied.

7. BEGINNER MISTAKE

✗ Using grep without quotes for text that contains spaces or special characters.

Wrong:

```
$ grep ERROR nginx error.log
```

Correct:

```
$ grep "ERROR" /var/log/nginx/error.log
```

2. COMMON GREP COMMANDS

Command	What it does	When to use it
grep	Searches for a pattern in a file.	Basic search (case sensitive).
grep -i	Case insensitive search.	When text may be UPPER or lower case.
grep -n	Shows line numbers in the output.	When you need to know where the match is located.
grep -r	Searches recursively in directories.	Search multiple files in a directory.
grep -E	Uses extended regular expressions (patterns).	When you need to search using patterns like multiple words or IPs.

4. SEARCH RECURSIVELY IN A DIRECTORY

```
# Search for a word in all files under /var/log
$ grep -r "error" /var/log
/var/log/nginx/error.log:2026/10/01 [error] failed
/var/log/syslog:Oct 01 10:21:15 server error detected
```

Use **-r** when you want to search through multiple files in a directory and its subdirectories.

6. PRACTICAL SCENARIOS

Find an IP address in a log file

```
$ grep "192.168.1.10" /var/log/nginx/access.log
192.168.1.10 - - [01/Oct/2026:10:20:01] "GET /index.html"
```

Find all ERROR lines and show line numbers

```
$ grep -n "ERROR" /var/log/nginx/error.log
15:2026/10/01 10:21:15 [error] upstream failed
```

Search multiple file types (e.g. .conf files)

```
$ grep -r "server_name" /etc/nginx/*.conf
/etc/nginx/nginx.conf: server_name example.com;
```

8. JUNIOR ENGINEER TIPS

- ✓ Use **-i** when you are not sure about the case (ERROR vs error).
- ✓ Use **-n** to quickly see the exact line number.
- ✓ Use **-r** to search multiple files in a directory.
- ✓ Use **-E** for flexible pattern searches.
- ✓ Combine **grep** with other commands like **less** or **tail** for better results (e.g. **grep "error" logfile | less**).



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Filter, Transform & Build Command Pipelines

1. WHY THIS MATTERS

- The Unix philosophy is to use small commands that do one job well and combine them.
- Pipelines (`|`) let you pass the output of one command to another.
- This helps you filter, transform and analyse data quickly.
- Administrators use pipelines to answer complex questions using simple tools.

2. HOW A PIPELINE WORKS



Each command does one thing and passes the result to the next command using the pipe (`|`).

3. KEY COMMANDS

Command	What it does	Example
sort	Sorts lines of text.	<code>sort access.log</code>
uniq	Removes duplicate lines (works best with sorted data).	<code>sort log uniq</code>
cut	Extracts specific columns or fields.	<code>cut -d' ' -f1 access.log</code>
tr	Translates or replaces characters.	<code>tr 'a-z' 'A-Z'</code>
awk	Powerful text processing tool.	<code>awk '{print \$1}' access.log</code>
sed	Search and replace text.	<code>sed 's/old/new/g' file.txt</code>
xargs	Builds and runs commands using input from another command.	<code>find . -name '*.log' xargs rm</code>
tee	Saves command output to a file while also showing it on screen.	<code>command tee output.txt</code>

4. PRACTICAL EXAMPLES

```
# 1. Count unique IP addresses in a log file
cat access.log | cut -d' ' -f1 | sort | uniq
192.168.1.10
192.168.1.11
192.168.1.12
```

```
# 2. Find and count error messages
grep "ERROR" app.log | sort | uniq -c
5 ERROR database connection failed
3 ERROR timeout reached
2 ERROR invalid user
```

```
# 3. Show only the first column (IP address)
cut -d' ' -f1 access.log | head
192.168.1.10
192.168.1.11
192.168.1.12
```

```
# 4. Convert text to uppercase
cat file.txt | tr 'a-z' 'A-Z'
WELCOME TO LINUX
COMMAND LINE TOOLKIT
```

5. REAL-WORLD SCENARIO

Find the top 5 most common IP addresses with failed logins:

```
grep "Failed password" /var/log/auth.log | \
cut -d' ' -f9 | sort | uniq -c | sort -nr | head -5
53 192.168.1.10
27 192.168.1.11
12 192.168.1.25
9 10.0.0.5
6 192.168.1.30
```

6. COMMON MISTAKES



- Using `uniq` without `sort` (results may be wrong).
- Using the wrong field number with `cut`.
- Not quoting patterns with spaces.
- Trying to solve everything with one complex command instead of simple pipelines.

7. JUNIOR ENGINEER TIPS

- ✓ Break big problems into smaller steps using pipelines.
- ✓ Use `head` to check output before running the full command.
- ✓ Use `tee` when you want to save and also view the output.
- ✓ Learn to combine these tools to analyse logs, files and data efficiently.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Copy, Move, Create & Remove Files

1. WHY THIS MATTERS

- As a Linux administrator, you will create, copy, move, delete and organise files and directories every day.
- These operations help you manage applications, configuration files, logs and backups.
- Using the correct command and options prevents data loss and mistakes.

3. CREATE FILES AND DIRECTORIES

```
# Create an empty file
```

```
$ touch file.txt
```

```
# Create multiple files
```

```
$ touch file1.txt file2.txt
```

```
# Create a new directory
```

```
$ mkdir projects
```

```
# Create nested directories
```

```
$ mkdir -p /data/logs/app
```

4. COPY FILES AND DIRECTORIES

```
# Copy a file
```

```
$ cp file.txt /backup/
```

```
# Copy and rename a file
```

```
$ cp file.txt /backup/file_backup.txt
```

```
# Copy a directory (recursive)
```

```
$ cp -r projects /backup/
```

5. MOVE AND RENAME FILES

```
# Move a file to a directory
```

```
$ mv file.txt /data/
```

```
# Rename a file
```

```
$ mv oldname.txt newname.txt
```

```
# Move and rename
```

```
$ mv file.txt /data/newfile.txt
```

8. COMMON MISTAKES



- Using **rm** or **rm -r** without checking the path.
- Accidentally deleting the wrong directory.
- Thinking **rm** moves files to a recycle bin.
- Using **rmdir** on a non-empty directory.
- Overwriting files with **cp** without realising.

2. KEY COMMANDS

Command	What it does	Example
touch	Creates an empty file or updates the timestamp.	touch file.txt
mkdir	Creates a new directory.	mkdir projects
cp	Copies files or directories.	cp file.txt /backup/
mv	Moves or renames files or directories.	mv file.txt /data/
rm	Removes files.	rm file.txt
rmdir	Removes empty directories.	rmdir old_dir
ln	Creates a link (symbolic or hard link).	ln -s /etc/nginx/nginx.conf nginx.conf

6. REMOVE FILES AND DIRECTORIES



BE VERY CAREFUL

- rm** permanently deletes files. It does not move them to a recycle bin.
- rm -r** deletes directories and everything inside them.
- rm -rf** forcefully and recursively deletes without confirmation. This can cause serious data loss.

```
# Remove a file
```

```
$ rm file.txt
```

```
# Remove multiple files
```

```
$ rm file1.txt file2.txt
```

```
# Remove an empty directory
```

```
$ rmdir old_dir
```

```
# Remove a directory and
```

```
all its contents
```

```
$ rm -r projects
```



SAFE VERIFICATION BEFORE DELETION

- Always check what you are about to delete.
- Use **ls -l** or **ls -lah** to confirm file names and paths.
- For large deletions, consider using **find** first to list the files.

```
# Preview files that would be deleted (safety check)
```

```
$ find /data/logs -type f -name "*.log"
```

Only run **rm** after you are sure the list is correct.

7. CREATE SYMBOLIC LINKS

```
# Create a symbolic link
```

```
$ ln -s /etc/nginx/nginx.conf nginx.conf
```

```
# Create a hard link
```

```
$ ln /etc/hosts hosts_backup
```

SYMBOLIC LINK EXAMPLE

- A symbolic link is like a shortcut that points to a file or directory.
- It is useful for application configuration, versioning and simplifying paths.

9. JUNIOR ENGINEER TIPS



- Use **ls -lah** to always confirm the file or directory exists.
- Use **find** to preview files before deleting many files.
- Use descriptive names when creating or renaming files.
- Use **-p** with **mkdir** to create parent directories automatically.
- Use symbolic links to manage configuration files safely.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Understand Ownership & Permissions

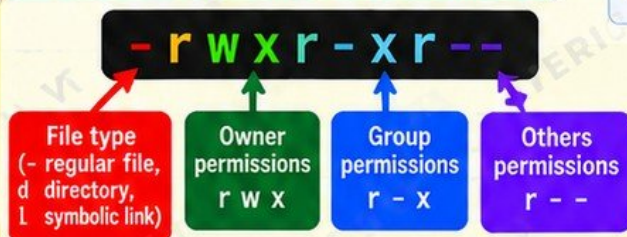
1. WHY THIS MATTERS

- Controls who can read, write and execute files and directories.
- Helps protect your system and data from unauthorised access.
- Required for applications, services and collaborative environments.
- Using the correct permissions prevents security issues and operational problems.

2. KEY COMMANDS

Command	What it does	Example
<code>ls -l</code>	Shows permissions, owner, group and details.	<code>ls -l /etc/nginx</code>
<code>chmod</code>	Changes file or directory permissions.	<code>chmod 755 script.sh</code>
<code>chown</code>	Changes file or directory owner.	<code>chown user:file.txt</code>
<code>chgrp</code>	Changes file or directory group.	<code>chgrp devops file.txt</code>
<code>stat</code>	Shows detailed file information.	<code>stat file.txt</code>
<code>umask</code>	Sets default permissions for new files and directories.	<code>umask 022</code>

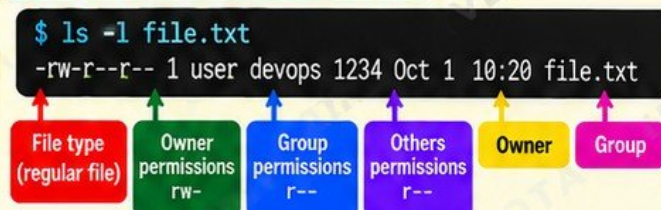
3. UNDERSTANDING PERMISSIONS



Permission meaning

r	Read	View file content or list directory
w	Write	Modify file content or create files
x	Execute	Run a file or enter a directory
-	No permission	

4. VIEW FILE PERMISSIONS



5. NUMERIC PERMISSIONS

Each permission has a number: r = 4, w = 2, x = 1

Permission	Binary	Decimal
r--	100	4
-w-	010	2
--x	001	1
r-x	101	5
rw-	110	6
rwX	111	7

Examples

```
$ chmod 755 script.sh
Owner: rwx (7)
Group : r-x (5)
Others: r-x (5)

$ chmod 644 file.txt
Owner: rw- (6)
Group : r-- (4)
Others: r-- (4)
```

6. CHANGE OWNERSHIP AND GROUP

Change file owner

```
$ sudo chown user file.txt
```

Change file group

```
$ sudo chgrp devops file.txt
```

Change owner and group together

```
$ sudo chown user:devops file.txt
```

7. CHECK DETAILED INFORMATION

```
$ stat file.txt
```

```
File: file.txt
Size: 1234      Blocks: 8      IO Block: 4096  regular file
Device: 802h/2050d Inode: 1234567 Links: 1
Access: 2026-10-01 10:20:15.000000000 +0000
Modify: 2026-10-01 10:20:15.000000000 +0000
Change: 2026-10-01 10:20:15.000000000 +0000
Birth: 2026-10-01 10:20:15.000000000 +0000
```

8. DEFAULT PERMISSIONS WITH UMASK

- `umask` sets the default permissions for new files and directories.
- Common values:
 - 022 → Files: 644, Directories: 755 (common)
 - 002 → Files: 664, Directories: 775 (group friendly)
 - 077 → Files: 600, Directories: 700 (restricted)

```
$ umask
022
```

9. COMMON MISTAKE



Using `chmod 777` everywhere.

- Gives full permission to everyone.
- Creates a security risk.
- Should not be used as a quick fix.
- Use the correct minimum permissions instead.

10. JUNIOR ENGINEER TIP



- Use the minimum permissions required.
- Understand who needs access (owner, group or others).
- Check permissions before changing them.
- Use human readable format (e.g. `chmod u+r file.txt`) when possible.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Users, Groups & Privilege

1. WHY THIS MATTERS

- Linux decides access based on users and groups.
- You must know who you are, what permissions you have, and who is logged in.
- Helps manage access, troubleshoot issues and maintain security.
- Use least privilege to reduce risks and prevent accidental changes.

2. KEY COMMANDS

Command	What it does	Example
whoami	Shows current username.	whoami
id	Shows UID, GID and groups.	id
who	Shows logged-in users.	who
w	Shows logged-in users and what they are doing.	w
groups	Shows groups for a user.	groups devops
getent	Reads account/group databases.	getent passwd nginx getent group sudo
sudo	Runs a command with elevated privileges.	sudo systemctl restart nginx

3. IDENTITY BASICS

```
$ id
uid=1001(devops) gid=1001(devops) groups=1001(devops),27(sudo),998(docker)
```

User ID (UID)

Identifies the user.
Example: 1001

Primary Group ID (GID)

Main group for the user.
Example: 1001

Supplementary Groups

Additional groups give extra access.
Example: sudo, docker

4. SEE WHO IS LOGGED IN

who – lists logged-in users

```
$ who
devops tty1 2026-10-01 10:15
alice pts/0 2026-10-01 11:22
bob pts/1 2026-10-01 11:20
```

Shows which users are currently logged in and their terminals.

w – shows users and activity

```
$ w
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
devops tty1 :0 10:15 2:10 0.01 0.00 bash
alice pts/0 192.168.1.10 11:02 5.00 0.10 0.05 vim
bob pts/1 203.0.113.5 11:20 1.00 0.02 0.01 top
```

Shows logged-in users, what they are doing and system load information.

5. CHECK GROUPS & ACCOUNT INFO

groups – shows groups for a user

```
$ groups devops
devops : devops sudo docker
```

getent passwd – shows account information

```
$ getent passwd nginx
nginx:x:998:996:NGINX user:/var/cache/nginx:/usr/sbin/nologin
```

getent group – shows group information

```
$ getent group sudo
sudo:x:27:devops,alice,bob
```

getent reads the system account and group databases (e.g. /etc/passwd and /etc/group).

6. USING SUDO SAFELY

```
$ sudo systemctl restart nginx
```

```
$ sudo cat /etc/shadow
root:!:19793:0:99999:7:::
alice:$6$...:19793:0:99999:7:::
```

- Use sudo only for commands that require elevated privileges.
- Do not stay logged in as root for routine work.
- Root has UID 0 and full access to the system.
- Example (safe): **sudo systemctl restart nginx**
- Example (powerful, use carefully): **sudo cat /etc/shadow**
- Always understand why a command needs elevated privileges.

7. COMMON BEGINNER MISTAKE



Working as root all the time.

- Can cause accidental destructive changes.
- Increases security exposure.
- Makes it hard to track who did what.
- Not following least privilege principles.

8. JUNIOR ENGINEER TIP



- Use a normal user for everyday work and elevate only when necessary.
- Least privilege reduces mistakes and improves security.
- Know which commands require sudo and understand why.
- Verify what a command does before running it with elevated privileges.

9. REAL-WORLD EXAMPLE

- A web server may require the user to be in the nginx group to access its files and directories.
- Use sudo for service restarts or configuration changes, not for every command.
- Check group membership when facing permission issues.
- Example: if a user is not in the docker group, they cannot run docker commands.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Processes: What Is Running Right Now?

1. WHY THIS MATTERS

- A process is a running program in Linux with its own ID, resources and owner.
- You can inspect running processes to understand system activity, troubleshoot performance issues and find problems.
- Helps identify which application is using high CPU or memory.
- Knowing what is running helps you maintain system stability and security.

3. PROCESS BASICS

PID	Process ID. Unique number for each process. Example: 1234
PPID	Parent Process ID. The process that started this process.
CPU	CPU usage of the process (percentage).
MEM	Memory usage of the process (percentage).
STAT	Process state (e.g. R running, S sleeping, D uninterruptible, Z zombie).
USER	Which user owns the process.
CMD	The command or program being executed.

2. KEY COMMANDS

Command	What it does	Example
ps	Shows current process snapshot.	ps
ps aux	Shows all processes with detailed information.	ps aux
pgrep	Finds process IDs by name.	pgrep nginx
top	Shows live process resource usage.	top
uptime	Shows system uptime and load averages.	uptime
pstree	Shows process relationships in a tree format.	pstree -p

4. PROCESS LINE EXAMPLE (ps aux)

```
$ ps aux | head -1
```

USER	PID	%CPU	%MEM	VSZ	RSS	STAT	START	TIME	COMMAND
root	1	0.0	0.1	16860	1056	Ss	Oct01	0:03	/sbin/init


USER
Process owner


PID
Process ID


CPU
CPU usage (%)


MEM
Memory usage (%)


STAT
Process state


START
When it started


TIME
Total CPU time used


COMMAND
Command being run

6. SLOW APPLICATION SCENARIO

- 1 User reports the application is slow.
- 2 Check system load and uptime.
\$ uptime
Look at load averages. High values may indicate resource pressure.
- 3 Find the process using high CPU or memory.
\$ top
Identify the process with high %CPU or %MEM.
- 4 Get detailed information about the process.
\$ ps aux | grep <name>
Check PID, user, memory, and command line.
- 5 Use **pgrep** to quickly find PIDs.
\$ pgrep <name>
- 6 Understand parent-child relationship.
\$ pstree -p | grep <name>
See what started the process.

5. PRACTICAL EXAMPLES

1. Basic process snapshot <pre>\$ ps</pre> <table> <tr> <th>PID</th> <th>TTY</th> <th>TIME</th> <th>CMD</th> </tr> <tr> <td>1234</td> <td>pts/0</td> <td>00:00:00</td> <td>bash</td> </tr> <tr> <td>5678</td> <td>pts/0</td> <td>00:00:00</td> <td>ps</td> </tr> </table>	PID	TTY	TIME	CMD	1234	pts/0	00:00:00	bash	5678	pts/0	00:00:00	ps	2. All processes with details <pre>\$ ps aux head -3</pre> <table> <tr> <th>USER</th> <th>PID</th> <th>%CPU</th> <th>%MEM</th> <th>VSZ</th> <th>RSS</th> <th>STAT</th> <th>COMMAND</th> </tr> <tr> <td>root</td> <td>1</td> <td>0.0</td> <td>0.1</td> <td>16860</td> <td>1056</td> <td>Ss</td> <td>/sbin/init</td> </tr> <tr> <td>root</td> <td>2</td> <td>0.0</td> <td>0.0</td> <td>0</td> <td>0</td> <td>S</td> <td>[kthreadd]</td> </tr> <tr> <td>alice</td> <td>1234</td> <td>0.1</td> <td>0.2</td> <td>57400</td> <td>4200</td> <td>S</td> <td>bash</td> </tr> </table>	USER	PID	%CPU	%MEM	VSZ	RSS	STAT	COMMAND	root	1	0.0	0.1	16860	1056	Ss	/sbin/init	root	2	0.0	0.0	0	0	S	[kthreadd]	alice	1234	0.1	0.2	57400	4200	S	bash
PID	TTY	TIME	CMD																																										
1234	pts/0	00:00:00	bash																																										
5678	pts/0	00:00:00	ps																																										
USER	PID	%CPU	%MEM	VSZ	RSS	STAT	COMMAND																																						
root	1	0.0	0.1	16860	1056	Ss	/sbin/init																																						
root	2	0.0	0.0	0	0	S	[kthreadd]																																						
alice	1234	0.1	0.2	57400	4200	S	bash																																						
3. Find process IDs by name <pre>\$ pgrep nginx</pre> <pre>998</pre> <pre>1021</pre> <pre>\$ ps aux grep nginx</pre> <pre>root 998 0.0 0.1 99812 4200 ? Ss 10:15 0:00 nginx: www-data 1021 0.5 1.2 215432 50200 ? S 10:15 0:02 worker process</pre>	4. Live process monitoring <pre>\$ top -b -n 1 head -5</pre> <pre>top - 10:16:24 up 1:23, 2 users, load average: 0.42, 0.38, 0.21 Tasks: 205 total, 1 running, 204 sleeping, 0 stopped, 0 zombie %Cpu(s): 2.1 us, 0.5 sy, 0.0 ni, 97.4 id, 0.0 wa, 0.0 hi, 0.0 si %Mem: 7829.2 total, 2120.5 used, 5708.7 free, 0.0 buff/cache</pre>																																												
5. System uptime and load averages <pre>\$ uptime</pre> <pre>10:16:24 up 1:23, 2 users, load average: 0.42, 0.38, 0.21</pre>	6. Process tree with PIDs <pre>\$ pstree -p head -5</pre> <pre>systemd(1) ├─nginx(998) │ └─nginx(1021) │ └─nginx(1022) └─sshd(712)</pre>																																												

7. COMMON BEGINNER MISTAKE

- ✗

Focusing only on the process name.
 - Killing a process without checking what it is.
 - Ignoring CPU and memory usage.
 - Not checking which user owns the process.
 - Not looking at the parent process.
 - This can cause accidental service outages.

8. JUNIOR ENGINEER TIP

- ✓

Use a step-by-step approach.
 - Start with **top** to quickly spot processes using high CPU or memory.
 - Use **ps aux** or **pgrep** to get detailed information.
 - Check the process owner and parent process with **pstree -p**.
 - Understand what the process is doing before taking action.
 - This helps you troubleshoot safely and avoid killing important processes.

Control Processes Without Making Things Worse

1. WHY THIS MATTERS

- You may need to stop, restart or manage processes when applications are unresponsive, using too many resources or misbehaving.
- Using the right signal and command helps you avoid data loss and service disruption.
- Understanding job control is useful when running long commands in your terminal.

3. UNDERSTAND LINUX SIGNALS

Signal	Number	What it does
SIGTERM	15	Graceful termination (default, safe).
SIGINT	2	Interrupt (Ctrl + C).
SIGKILL	9	Force kill (immediate, no cleanup).
SIGHUP	1	Reload or restart process.

4. BASIC EXAMPLES

```
# 1. Stop a process gracefully (send SIGTERM)
$ kill 12345

# 2. Force stop a process (use only if needed)
$ kill -9 12345

# 3. Stop processes by name
$ pkill nginx

# 4. Stop all processes with a name
$ killall nginx
```

7. TROUBLESHOOTING SCENARIO

A process appears frozen and the application is not responding. You need to stop it without making things worse.

```
# 1. Check the process
$ ps aux | grep app

# 2. Try graceful termination first
$ kill 12345

# 3. If it does not stop, use force (last option)
$ kill -9 12345
```

2. KEY COMMANDS

Command	What it does	When to use it
kill	Sends a signal to a process using its PID.	Stop a specific process gracefully or forcefully.
pkill	Sends a signal to processes by name (pattern).	Stop processes by name.
killall	Sends a signal to all processes with the exact name.	Stop all instances of a process.
nice	Starts a process with lower or higher priority.	Run a new process with a specific priority.
renice	Change the priority of a running process.	Adjust CPU priority without stopping it.
jobs	List background jobs in your current shell.	Manage background processes.
bg	Resume a job in the background.	Continue a stopped job.
fg	Bring a background job to the foreground.	Work with a background job in the terminal.

5. CONTROL PROCESS PRIORITY

```
# Start a process with lower priority (less CPU)
$ nice -n 10 ./backup.sh

# Change priority of a running process
$ renice 10 -p 12345
```

Higher nice value = lower priority (uses less CPU).
Use for non-critical tasks.

6. JOB CONTROL IN THE TERMINAL

```
# Run a long command and send it to background
$ sleep 3600 &

# List background jobs
$ jobs

# Bring a job to foreground
$ fg 1

# Send a job to background
$ bg 1
```

Useful for long running tasks so you can continue working in your terminal.

8. COMMON MISTAKES



- Using **kill -9** as the first option.
- Killing the wrong PID.
- Stopping system processes.
- Using **pkill** with a broad pattern (e.g. **pkill java**).
- Not checking what the process does before killing it.
- Forgetting that killing a process may cause data loss or service downtime.

9. JUNIOR ENGINEER TIPS



- Always check what the process is before killing it.
- Try graceful termination first (kill or Ctrl + C).
- Use top or ps to confirm the process and its resource usage.
- Be careful with **pkill** and **killall**.
- Use **nice** and **renice** for non-critical tasks.
- Use job control to manage long running commands safely.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Manage Services with systemd

1. WHY THIS MATTERS

- Services (daemons) run in the background to provide functionality such as web servers, databases and monitoring agents.
- systemd is the default init system on most modern Linux distributions.
- You need to manage services to keep systems stable, secure and available.
- Understanding start, stop, restart, reload and enable helps you troubleshoot issues and control services correctly.

2. KEY COMMANDS

Command	What it does	Example
<code>systemctl status</code>	Shows the current status of a service.	<code>systemctl status nginx</code>
<code>systemctl start</code>	Starts a service.	<code>systemctl start nginx</code>
<code>systemctl stop</code>	Stops a service.	<code>systemctl stop nginx</code>
<code>systemctl restart</code>	Stops and starts a service (used when configuration changes require full restart).	<code>systemctl restart nginx</code>
<code>systemctl reload</code>	Reloads the service configuration without full restart (if supported).	<code>systemctl reload nginx</code>
<code>systemctl enable</code>	Enables a service to start at boot time.	<code>systemctl enable nginx</code>
<code>systemctl disable</code>	Disables a service from starting at boot time.	<code>systemctl disable nginx</code>
<code>systemctl is-active</code>	Checks if a service is running.	<code>systemctl is-active nginx</code>
<code>systemctl is-enabled</code>	Checks if a service is enabled at boot.	<code>systemctl is-enabled nginx</code>

3. WHAT IS A SERVICE?

- A service is a managed background process.
- Each service is defined by a unit file (e.g. `nginx.service`).
- Services can be started, stopped, reloaded and enabled to start automatically at boot.
- Common unit types: `.service` (main), `.timer`, `.socket`, `.target`, `.mount`.

4. CHECK SERVICE STATUS

```
# Check if nginx is running
$ systemctl status nginx

● nginx.service - A high performance web server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled)
   Active: active (running) since Wed 2026-10-01 10:21:15 UTC
     Main PID: 1234 (nginx)
       Tasks: 3 (limit: 4612)
      Memory: 12.3M
     CGroup: /system.slice/nginx.service
             └─1234 nginx: master process /usr/sbin/nginx
               └─1235 nginx: worker process
                 └─1236 nginx: worker process
```

5. START, STOP, RESTART AND RELOAD

```
# Start a service
$ systemctl start nginx

# Stop a service
$ systemctl stop nginx

# Restart a service (full restart)
$ systemctl restart nginx

# Reload a service (re-read configuration, no full restart)
$ systemctl reload nginx
```

Restart vs Reload

- Restart stops and starts the service.
- Use when major changes are made.
- Reload re-reads the configuration without stopping the service (if supported).
- Faster and causes less disruption.

6. ENABLE OR DISABLE AT BOOT

```
# Enable service to start at boot
$ systemctl enable nginx

# Disable service from starting at boot
$ systemctl disable nginx

# Check if service is enabled
$ systemctl is-enabled nginx
enabled

# Check if service is active (running)
$ systemctl is-active nginx
active
```

7. TROUBLESHOOTING SCENARIO

An application is unavailable. You need to check the service, find the reason and fix it.

```
# 1. Check service status
$ systemctl status nginx

# 2. Check recent logs for errors
$ journalctl -u nginx -n 50

# 3. Check configuration for syntax errors
$ nginx -t

# 4. If you made a config change, reload or restart
$ systemctl reload nginx # or $ systemctl restart nginx
```

8. COMMON MISTAKES



- Restarting the service when a simple reload is enough.
- Forgetting to enable the service at boot.
- Not checking logs when the service fails.
- Editing configuration files without testing syntax.
- Disabling a service by mistake.

9. JUNIOR ENGINEER TIPS



- Always check the service status and logs first.
- Use reload instead of restart when supported.
- Verify configuration syntax before restarting.
- Enable important services to start at boot.
- Understand the difference between active and enabled.
- Never disable a service without a reason.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Investigate Logs with journalctl

1. WHY THIS MATTERS

- Logs are **evidence**. They tell you what happened, when it happened and why.
- journalctl** reads logs from the systemd journal, which collects logs from the kernel, services and applications.
- Investigating logs helps you find the real cause of a problem instead of blindly restarting the service.
- Good log analysis leads to faster and safer problem resolution.

2. KEY COMMANDS

Command	What it does	Example
journalctl	Shows all system logs (from the systemd journal).	journalctl
journalctl -u	Shows logs for a specific service (unit).	journalctl -u nginx
journalctl -f	Follows logs in real time (live view).	journalctl -u nginx -f
journalctl -b	Shows logs from the current boot.	journalctl -b
journalctl --since	Shows logs since a specific time or date.	journalctl --since "1 hour ago"

3. UNDERSTANDING journalctl

- journalctl** reads structured logs stored by systemd in `/var/log/journal`.
- Logs include timestamps, service name, process ID and messages.
- Use **-u** to investigate a specific service.
- Use **-f** to watch new logs as they happen.
- Use **-b** to view logs from the current boot.
- Use **--since** to filter logs by time.

5. INVESTIGATION WORKFLOW

- Check service status**
`$ systemctl status nginx`
Confirm the service is failed or not running.
- Read the logs**
`$ journalctl -u nginx -n 50`
Look for errors and the reason for the failure.
- Analyse the cause**
Identify the error, misconfiguration or missing dependency.
- Fix the issue**
Update configuration, fix permissions or install missing components.
- Verify the service**
`$ systemctl restart nginx`
`$ systemctl status nginx`
Confirm the service is running and check the logs again.

7. COMMON MISTAKES



- Restarting the service without checking the logs.
- Ignoring error messages in the logs.
- Only looking at the last few lines.
- Not filtering logs by time during investigations.
- Thinking the service is broken without checking configuration and dependencies.
- Clearing logs instead of investigating the cause.
- Not checking system logs when the service fails.

4. PRACTICAL EXAMPLES

- View all system logs**
`$ journalctl`
Shows recent logs from the system and all services.
- View logs for a specific service**
`$ journalctl -u nginx`
Shows only logs for the nginx service.
- Follow logs in real time**
`$ journalctl -u nginx -f`
Live view of new logs (use Ctrl + C to stop).
- View logs from current boot**
`$ journalctl -b`
Shows logs since the last boot.
- View logs since a specific time**
`$ journalctl --since "1 hour ago"`
Shows logs from the last 1 hour (use other time formats too).
- View the last 50 lines**
`$ journalctl -u nginx -n 50`
Shows the most recent 50 log lines.

6. REAL-WORLD SCENARIO

An application service is not available. Investigate the logs to find the cause instead of restarting it immediately.

```
# 1. Check service status
$ systemctl status nginx
# 2. View recent logs
$ journalctl -u nginx -n 50
# Sample log output (error)
Oct 01 10:20:15 server nginx[1234]: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
Oct 01 10:20:15 server nginx[1234]: [emerg] still could not bind()
# 3. Find what is using the port (fix the issue)
$ ss -tln | grep :80
# 4. After fixing, restart and verify
$ systemctl restart nginx
$ systemctl status nginx
$ journalctl -u nginx -n 20
```

8. JUNIOR ENGINEER TIPS



- Always check `systemctl status` first.
- Use **journalctl** to understand the real error.
- Look for clear error messages and timestamps.
- Use **-n** to limit output and **--since** to filter time.
- Fix the actual cause instead of repeatedly restarting.
- After making changes, verify the service and check the logs again.
- Logs are your best friend when troubleshooting.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Understand IP Addresses & Network Interfaces

1. WHY THIS MATTERS

- Helps you understand how your server connects to the network and other systems.
- You will work with IP addresses, interfaces and routes every day as a DevOps or cloud engineer.
- Knowing the correct information helps you troubleshoot connectivity, service availability and network issues.

2. KEY COMMANDS

Command	What it does	Example
<code>ip addr</code>	Shows IP addresses assigned to network interfaces.	<code>ip addr</code>
<code>ip link</code>	Shows network interfaces and their status.	<code>ip link</code>
<code>ip route</code>	Shows the routing table (default route and other routes).	<code>ip route</code>
<code>hostname</code>	Shows the current hostname.	<code>hostname</code>
<code>hostnamectl</code>	Shows or sets the system hostname and related details.	<code>hostnamectl</code>

3. UNDERSTANDING NETWORK INTERFACES

```
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 state UNKNOWN
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
   inet 192.168.1.10/24 brd 192.168.1.255 scope global eth0
       valid_lft forever preferred_lft forever
```

Interface name
(lo, eth0)

Flags and status
(UP, DOWN)

IP address
(127.0.0.1, 192.168.1.10)

Subnet
(/8, /24)

4. COMMON INTERFACES

Interface	Description	Typical Use
lo	Loopback interface (127.0.0.1)	Used for local communication on the same server.
eth0 / ens3	Physical network interface (varies by system)	Connects to the network (LAN or internet).
docker0	Docker bridge interface	Used by Docker containers.
veth*	Virtual interface	Used by containers or VMs.
tun0 / wg0	VPN interface	Used for VPN connections.

5. UNDERSTANDING IP ADDRESSES

Type	Example	Description
Loopback	127.0.0.1	Refers to the local machine. Used for local testing only. Not reachable from other machines.
Private IP	192.168.1.10 10.0.0.5 172.16.0.10	Used within a private network (LAN). Not directly accessible from the internet.
Public IP	203.0.113.25	Globally unique and accessible from the internet.

6. VIEW AND UNDERSTAND ROUTES

```
$ ip route
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link
10.0.0.0/8 dev tun0 proto kernel scope link
```

Default route
(traffic to internet)

Network route
(local network)

Interface used
(eth0, tun0)

7. CHECK AND SET HOSTNAME

```
# Check current hostname
```

```
$ hostname
web-server-01
```

```
# View detailed hostname information
```

```
$ hostnamectl
Static hostname: web-server-01
Icon name: computer-vm
Operating System: Ubuntu 22.04 LTS
Kernel: Linux 5.15.0-102-generic
Architecture: x86_64
```

```
# Set a new hostname (requires sudo)
```

```
$ sudo hostnamectl set-hostname web-server-02
```

8. WHY YOU CANNOT INTERCHANGE IP ADDRESSES



127.0.0.1 cannot be used instead of a private or public IP.

- It only works on the same machine.
- Other machines cannot reach 127.0.0.1.



A private IP cannot be used instead of a public IP.

- Private IPs are not routed on the internet.
- They are only reachable within the local network.



A public IP cannot be used as 127.0.0.1 or a private IP.

- A public IP is used to reach the server from the internet.
- It is not the same as local or private addressing.

9. JUNIOR ENGINEER TIPS



- Use `ip addr` to check which IP address is assigned to each interface.
- Use `ip route` to find the default gateway.
- Always understand whether you are using a private or public IP.
- Do not use 127.0.0.1 to test services that need to be accessed from other machines.
- Verify the hostname to easily identify your servers.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Troubleshoot DNS & Connectivity

1. WHY THIS MATTERS

- Helps you identify where a connectivity problem is happening: DNS, network, routing or the application.
- Saves time by following a clear troubleshooting flow.
- Prevents unnecessary restarts and wrong fixes.
- Essential for working with servers, cloud services and external systems.

2. KEY COMMANDS

Command	What it does	Example
ping	Tests basic network reachability (ICMP).	ping google.com
dig	Queries DNS records.	dig veriqta.com
getent hosts	Checks hostname resolution using system configuration.	getent hosts google.com
curl	Tests an application request (HTTP/HTTPS).	curl -I https://google.com
traceroute or tracepath	Shows the network path to the destination.	traceroute google.com or tracepath google.com

3. UNDERSTANDING DNS

- DNS (Domain Name System) converts a domain name (e.g. google.com) to an IP address (e.g. 142.250.72.14).
- If DNS fails, the hostname cannot be resolved, even if the server is reachable.
- DNS is different from network connectivity. You can have network access but still have DNS issues.

4. TROUBLESHOOTING FLOW



5. EXAMPLES AND OUTPUT

Check DNS resolution with dig

```
$ dig google.com
;; ANSWER SECTION:
google.com. 300 IN A
142.250.72.14
```

Check DNS resolution with getent

```
$ getent hosts google.com
142.250.72.14 google.com
```

Test network reachability with ping

```
$ ping -c 4 google.com
PING google.com (142.250.72.14)
56(84) bytes of data:
4 packets transmitted, 4 received,
0% packet loss, time 3003ms
```

Check network path with traceroute

```
$ traceroute google.com
traceroute to google.com
1 192.168.1.1 0.8 ms
2 10.0.0.1 12.4 ms
3 142.250.72.14 18.6 ms
```

Test application with curl

```
$ curl -I https://google.com
HTTP/2 200
content-type: text/html; charset=UTF-8
server: gws
content-length: 172354
...
```

What this tells you

- ✓ dig / getent → DNS works
- ✓ ping → network reachability works
- ✓ traceroute → shows the path
- ✓ curl → application is responding

6. COMMON ISSUES AND HOW TO IDENTIFY

Issue	Symptoms	How to check	Possible cause
DNS problem	Cannot resolve hostname	dig, getent hosts	Wrong DNS server, missing DNS record
Network problem	Cannot reach IP address	ping, traceroute	Firewall, routing issue, network down
Application problem	Can reach IP but application fails	curl	Service not running, wrong port, application error

Key point Each step helps you find exactly where the problem is instead of guessing.

7. COMMON MISTAKES

- ✗ Restarting the service without checking DNS or network.
- ✗ Assuming a DNS problem is a network problem.
- ✗ Using ping to test an application (ping only tests ICMP).
- ✗ Not checking the actual IP address being used.
- ✗ Ignoring firewall or security group rules.
- ✗ Only testing from one machine.

8. JUNIOR ENGINEER TIPS

- ✓ Always check DNS resolution first.
- ✓ Test both the domain name and the IP address.
- ✓ Use traceroute to identify where the connection stops.
- ✓ Use curl to verify the actual application response.
- ✓ Compare results from another server or network if possible.
- ✓ Document what you find at each step.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Ports, Sockets & Listening Services

1. WHY THIS MATTERS

- Helps you understand how applications listen for connections and how clients connect to them.
- Useful for troubleshooting when a service is running but users cannot connect (e.g. Connection refused).
- Helps you identify which process is using a port and whether it is exposed to the network.
- Important for security, service availability and proper configuration.

2. KEY COMMANDS

Command	What it does	Example
<code>ss -tulpn</code>	Shows all listening ports (TCP and UDP) with process information.	<code>ss -tulpn</code>
<code>ss -ltnp</code>	Shows listening TCP ports with process information (only TCP).	<code>ss -ltnp</code>
<code>curl</code>	Tests a connection to a service using HTTP or HTTPS.	<code>curl http://localhost:8080</code>
<code>nc</code>	Tests a connection to a port using netcat.	<code>nc -zv localhost 8080</code>

3. UNDERSTANDING PORTS, SOCKETS AND PROTOCOLS

- An **IP address** identifies a device (e.g. 192.168.1.10).
- A **port** identifies a specific service on that device (e.g. port 80 for HTTP, port 22 for SSH).
- A **protocol** (TCP or UDP) defines how data is sent.
- A **listening socket** means a service is waiting for incoming connections on a specific IP and port.
- A **connection** can be in listening (waiting) or established (active) state.

4. COMMON PORTS (EXAMPLES)

Port	Service	Protocol	Common Use
22	SSH	TCP	Remote server access
80	HTTP	TCP	Web traffic (not encrypted)
443	HTTPS	TCP	Web traffic (encrypted)
3306	MySQL	TCP	Database access
5432	PostgreSQL	TCP	Database access
6379	Redis	TCP	Cache service
8080	Application	TCP	Common for web applications

5. FIND LISTENING PORTS AND PROCESSES

Show all listening ports (TCP and UDP)

```
$ ss -tulpn
Netid State Local Address:Port Process
tcp LISTEN 0.0.0.0:22 users:(sshd,pid=1234)
tcp LISTEN 127.0.0.1:3306 users:(mysqld,pid=2345)
tcp LISTEN 0.0.0.0:8080 users:(java,pid=3456)
```

Show listening TCP ports only

```
$ ss -ltnp
State Local Address:Port Process
LISTEN 0.0.0.0:22 users:(sshd,pid=1234)
LISTEN 127.0.0.1:3306 users:(mysqld,pid=2345)
LISTEN 0.0.0.0:8080 users:(java,pid=3456)
```

6. TEST A CONNECTION

Test using curl (HTTP/HTTPS)

```
$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Server: nginx
Date: Wed, 01 Oct 2026 10:00:00 GMT
...
```

If the service is working, you should get a response (e.g. 200 OK). If it fails, you will see an error.

Test using nc (netcat)

```
$ nc -zv 192.168.1.10 8080
Connection to 192.168.1.10 8080 port [tcp/*] succeeded!

$ nc -zv 192.168.1.10 8080
nc: connect to 192.168.1.10 port 8080 (tcp) failed:
Connection refused
```

Connection refused means nothing is listening on that IP and port.

7. TROUBLESHOOTING SCENARIO

The application process is running, but users receive Connection refused when trying to access it.

- Check if the process is running `$ ps aux | grep app`
- Check if the port is listening `$ ss -ltnp | grep :8080`
- Test locally `$ curl http://localhost:8080`
- Test from another server `$ curl http://192.168.1.10:8080`
- Check firewall rules `$ sudo iptables -L -n | grep 8080`
or use `firewall-cmd / ufw`
- Check service configuration # Verify correct IP and port in the application config file.

8. COMMON MISTAKES



- Assuming the application is listening on all interfaces.
- Trying to connect to the wrong IP or port.
- Not checking if the port is actually open and listening.
- Ignoring firewall or security group rules.
- Using the wrong protocol (HTTP vs HTTPS).
- Assuming the service is accessible externally by default.
- Not verifying the application configuration file.

9. JUNIOR ENGINEER TIPS



- Always check if the port is listening before testing.
- Use localhost first, then the server IP, then from another server.
- Verify you are using the correct IP, port and protocol.
- Check firewall or security group rules if the service is not reachable.
- Use ss instead of older tools like netstat.
- Confirm the application is configured to listen on the correct interface (0.0.0.0 to accept external connections).



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Disk Space & Filesystem Investigation

1. WHY THIS MATTERS

- A full disk can cause applications to fail, databases to stop, and the system to become unstable.
- You need to quickly find what is using space and understand the filesystem layout.
- Proper investigation prevents accidental deletion of important files.
- Essential skill for Linux administrators, DevOps and SRE engineers.

2. KEY COMMANDS

Command	What it does	Example
<code>df -h</code>	Shows filesystem disk usage in human readable format.	<code>df -h</code>
<code>du -sh</code>	Shows the total size of a directory.	<code>du -sh /var/log</code>
<code>du -xh</code>	Shows size of directory (excluding other filesystems).	<code>du -xh /var</code>
<code>lsblk</code>	Lists block devices and their mount points.	<code>lsblk</code>
<code>findmnt</code>	Shows mounted filesystems and their details.	<code>findmnt</code>
<code>mount</code>	Shows currently mounted filesystems.	<code>mount</code>

3. UNDERSTANDING DISK SPACE AND FILESYSTEMS

- A filesystem is a way to organise and store files on a block device (e.g. /, /home, /var).
- A block device is a physical or virtual disk (e.g. /dev/sda1).
- `df` shows overall filesystem usage (total, used, available).
- `du` shows which directories and files are consuming space.
- A server can be "full" because of logs, application data, container images, databases or user files.

4. CHECK DISK USAGE WITH df

```
# Check filesystem disk usage
```

```
$ df -h
```

Filesystem	Size	Used	Avail	Use%	Mounted on
udev	3.9G	0	3.9G	0%	/dev
tmpfs	795M	1.2M	794M	1%	/run
/dev/root	30G	28G	1.5G	95%	
tmpfs	3.9G	0	3.9G	0%	/dev/shm
/dev/sda1	500M	6.1M	494M	2%	/boot
/dev/sdb1	100G	45G	55G	45%	/data

Use% shows how full the filesystem is.

5. FIND WHAT IS USING SPACE WITH du

```
# Check size of a directory
```

```
$ du -sh /var/log
```

```
12G /var/log
```

Shows total size of the directory in a human readable format.

```
# Find largest directories
```

```
$ du -xh /var | sort -h
```

```
500M /var/cache
2.1G /var/lib
8.5G /var/log
12G /var
```

Shows directory sizes excluding other filesystems (useful for /var).

6. VIEW BLOCK DEVICES WITH lsblk

```
# List block devices and mount points
```

```
$ lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPPOINTS
sda	8:0	0	100G	0	disk	
└─sda1	8:1	0	500M	0	part	/boot
└─sda2	8:2	0	99.5G	0	part	/
sdb	8:16	0	100G	0	disk	
└─sdb1	8:17	0	100G	0	part	/data

Shows disks, partitions and where they are mounted.

7. VIEW MOUNTED FILESYSTEMS WITH findmnt

```
# Show mounted filesystems
```

```
$ findmnt
```

TARGET	SOURCE	FSTYPE	OPTIONS
/	/dev/sda2	ext4	rw,relatime
/boot	/dev/sda1	ext4	rw,relatime
/data	/dev/sdb1	ext4	rw,relatime
/run	tmpfs	tmpfs	rw,nosuid,nodev
/var/log	/dev/sdc1	xfs	rw,relatime

Shows all mounted filesystems with their source device, type and options.

8. INVESTIGATION FLOW (SERVER FULL)

1	Check overall disk usage	<code>\$ df -h</code> (identify full filesystem)
2	Find which directory is large	<code>\$ du -xh / sort -h</code>
3	Investigate inside the directory	<code>\$ du -sh /var/log/*</code> (example)
4	Identify the cause	Check logs, application data, container images, backups etc.
5	Take action	Rotate logs, clean old files, move data, or increase disk space.
6	Verify	<code>\$ df -h</code> (confirm space is freed)

9. COMMON MISTAKES



- Deleting files without understanding what they are.
- Only checking `df` but not using `du` to find the actual cause.
- Ignoring logs, cache, container images and temporary files.
- Not checking other filesystems (e.g. /var, /home, /data).
- Assuming the issue is the root filesystem only.



10. JUNIOR ENGINEER TIPS

- Always start with `df` to identify which filesystem is full.
- Use `du` to find the exact directory consuming space.
- Check logs, backups, container data and application files.
- Never delete files you do not recognise.
- Consider setting up log rotation and monitoring disk usage.
- If space keeps running out, plan to increase disk size or clean old data.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Memory & CPU Troubleshooting

1. WHY THIS MATTERS

- High memory or CPU usage can cause application latency, slow responses, timeouts and even service failures.
- Knowing how to check memory, swap, CPU and running processes helps you identify the real cause.
- You make better decisions when you use evidence instead of guessing.
- Essential skill for Linux administrators, DevOps and SRE engineers.

2. KEY COMMANDS

Command	What it does	Example
free -h	Shows memory usage (including swap) in a human readable format.	free -h
vmstat	Shows system performance (memory, CPU, IO, swap).	vmstat 1 5
top	Shows real time CPU and memory usage by processes.	top
ps	Shows currently running processes and their resource usage.	ps aux --sort=-%mem ps aux --sort=-%cpu

3. UNDERSTANDING MEMORY

```
$ free -h
              total        used        free      shared  buff/cache   available
Mem:    7.7G        6.1G        512M        200M        1.1G        1.2G
Swap:   2.0G        1.5G        512M
```

total	used	free
Total physical memory available on the server.	Memory currently in use by processes, cache and system.	Memory not in use (ready for new processes).
buff/cache	available	swap
Used for disk cache (improves performance).	Memory available for new applications (important value).	Virtual memory on disk used when physical memory is not enough.

4. CHECK SYSTEM PERFORMANCE WITH vmstat

```
$ vmstat 1 5
procs -----memory----- --swap-- --io-- --system-- --cpu-----
r b swpd free buff cache si so bi bo in ms us sy id wa
2 0 1536 512000 120000 980000 0 0 50 120 200 400 15 5 78 2
4 0 1536 480000 118000 990000 0 0 70 160 300 500 35 10 52 3
1 0 1536 500000 119000 995000 0 0 40 100 180 350 18 6 74 2
```

r	b	swpd	us	sy	id	wa
Processes waiting for CPU.	Processes in uninterruptible sleep (usually IO).	Amount of swap in use.	CPU used by user processes.	CPU used by system (kernel).	CPU idle (available).	CPU waiting for IO (disk, network).

5. CHECK CPU USAGE WITH top

```
$ top
top - 10:20:15 up 1:02, 1 user, load average: 2.35, 2.10, 1.75
Tasks: 246 total, 2 running, 244 sleeping, 0 stopped, 0 zombie
%Cpu(s): 35.2 us, 10.1 sy, 0.0 ni, 54.1 id, 0.6 wa, 0.0 hi, 0.0 si
MiB Mem : 7882.0 total, 512.0 free, 6123.0 used, 1247.0 buff/cache
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	COMMAND
1234	app	20	0	2.1g	512m	100m	R	85.3	6.5	java
2345	root	20	0	500m	120m	50m	S	12.1	1.5	nginx
3456	mysql	20	0	1.5g	800m	200m	S	10.2	10.1	mysqld

Shows real time CPU and memory usage by processes.
Press q to exit. Use top to quickly identify high resource processes.

7. TROUBLESHOOTING SCENARIO

Application latency rises and users report slow responses.

1	Check system load	\$ uptime
2	Check CPU usage	\$ top
3	Check memory usage	\$ free -h
4	Check performance trend	\$ vmstat 1 5
5	Identify top processes	\$ ps aux --sort=-%cpu head \$ ps aux --sort=-%mem head
6	Investigate further	Check application logs, process behavior and recent changes.

6. FIND RESOURCE CONSUMING PROCESSES WITH ps

Top 5 processes by memory usage

```
$ ps aux --sort=-%mem | head -n 6
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
app	1234	85.3	6.5	2147488	524288	?	R	10:10	5:10	java
mysql	3456	10.2	10.1	1572864	819200	?	S	09:30	1:20	mysqld
root	2345	12.1	1.5	512000	122880	?	S	09:45	0:50	nginx
root	5678	2.3	0.8	300000	65536	?	S	09:50	0:30	python
root	6789	1.1	0.5	250000	40960	?	S	09:55	0:10	cron

Use ps to find processes using the most CPU or memory.
You can also use: ps aux --sort=-%cpu (top CPU processes).

8. COMMON CAUSES

Issue	Symptoms	Possible Cause
High CPU usage	High %CPU, high load average, slow response	CPU intensive process, runaway process, infinite loop
High memory usage	Low available memory, increased swap usage	Memory leak, large cache, high load, too many processes
High swap usage	System becomes very slow	Not enough RAM, memory leak, high memory demand
High load average	Many processes waiting	High CPU, disk IO or network IO
Application latency	Users get slow response	High CPU or memory usage, database, external dependency

9. JUNIOR ENGINEER TIPS



- Always check memory and CPU before restarting a service.
- Use both top and ps to identify the exact process.
- Look at load average to understand system pressure.
- Check swap usage if the system is slow.
- Investigate the cause before taking action.
- Monitor regularly to prevent issues.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

Packages, Archives & Software Management

1. WHY THIS MATTERS

- Helps you install, update, remove and manage software safely.
- Different Linux distributions use different package managers and package formats.
- Understanding package management prevents compatibility issues and system instability.
- Archives and compression are essential for backups, transfers and deployments.

2. KEY COMMANDS

Command	What it does	Example
apt	Package manager for Debian and Ubuntu systems.	<code>sudo apt install nginx</code>
dnf	Package manager for RHEL 8+, Fedora and CentOS Stream.	<code>sudo dnf install nginx</code>
rpm	Package format and tool for RHEL, CentOS and Fedora.	<code>rpm -ivh package.rpm</code>
dpkg	Package format and tool for Debian and Ubuntu.	<code>sudo dpkg -i package.deb</code>
tar	Creates and extracts archive files.	<code>tar -xvf file.tar</code>
gzip	Compresses and decompresses files.	<code>gzip file.txt</code>

3. PACKAGE MANAGERS vs PACKAGE FORMATS

Package Manager (High-level tool)	Package Format (Low-level file format)
- Downloads packages from repositories. - Handles dependencies automatically. - Used to install, update, remove and manage software. - Examples: apt (Debian/Ubuntu), dnf (RHEL/Fedora).	- Actual package file that contains software. - Can be installed manually. - Does not automatically resolve dependencies. - Examples: .deb (used with dpkg), .rpm (used with rpm).

4. PACKAGE MANAGER EXAMPLES

Distribution	Package Manager	Common Commands
Ubuntu (Debian)	apt	<code>sudo apt update</code> <code>sudo apt install <package></code> <code>sudo apt remove <package></code>
RHEL / CentOS (Fedora)	dnf	<code>sudo dnf install <package></code> <code>sudo dnf remove <package></code> <code>sudo dnf update</code>
Older RHEL / CentOS	yum	<code>sudo yum install <package></code> <code>sudo yum remove <package></code>
All RPM based	rpm	<code>rpm -ivh <package.rpm></code> <code>rpm -e <package></code>
All Debian based	dpkg	<code>sudo dpkg -i <package.deb></code> <code>sudo dpkg -r <package></code>

5. COMMON PACKAGE MANAGEMENT TASKS

Ubuntu / Debian (apt)	RHEL / CentOS / Fedora (dnf)
<code># Update package list</code> <code>sudo apt update</code>	<code># Install a package</code> <code>sudo dnf install nginx</code>
<code># Install a package</code> <code>sudo apt install nginx</code>	<code># Remove a package</code> <code>sudo dnf remove nginx</code>
<code># Remove a package</code> <code>sudo apt remove nginx</code>	<code># Update all packages</code> <code>sudo dnf update</code>
<code># Upgrade all packages</code> <code>sudo apt upgrade</code>	<code># Search for a package</code> <code>sudo dnf search nginx</code>

6. USING rpm AND dpkg DIRECTLY

rpm (RHEL/CentOS/Fedora)	dpkg (Ubuntu/Debian)
<code># Install an RPM package</code> <code>sudo rpm -ivh package.rpm</code>	<code># Install a DEB package</code> <code>sudo dpkg -i package.deb</code>
<code># Query installed package</code> <code>rpm -q nginx</code>	<code># Query installed package</code> <code>dpkg -l nginx</code>
<code># List package files</code> <code>rpm -ql nginx</code>	<code># List package files</code> <code>dpkg -L nginx</code>
<code># Remove a package</code> <code>sudo rpm -e nginx</code>	<code># Remove a package</code> <code>sudo dpkg -r nginx</code>

7. ARCHIVES AND COMPRESSION

Command	What it does	Example
tar -cvf	Create an archive.	<code>tar -cvf app.tar /var/app</code>
tar -xvf	Extract an archive.	<code>tar -xvf app.tar</code>
tar -czvf	Create a compressed archive (gzip).	<code>tar -czvf app.tar.gz /var/app</code>
tar -xzvf	Extract a gzip archive.	<code>tar -xzvf app.tar.gz</code>
gzip	Compress a single file.	<code>gzip file.txt</code>
gunzip	Decompress a gzip file.	<code>gunzip file.txt</code>

8. COMMON MISTAKES

- Using the wrong package manager for your distribution.
- Trying to install an RPM package on Ubuntu or a DEB package on RHEL without the correct tool.
- Ignoring dependency errors.
- Downloading software from random sources instead of official repositories.
- Extracting archives to the wrong location.
- Deleting important files after extracting archives.

9. JUNIOR ENGINEER TIPS

- Always confirm your Linux distribution before using a package manager.
- Use official repositories whenever possible.
- Read package information before installing.
- Check disk space before installing large packages.
- Use archives (tar + gzip) for backups and file transfers.
- Keep your system updated for security and stability.



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

SSH & Remote Server Administration

1. WHY THIS MATTERS

- Allows you to securely access and manage remote servers.
- SSH keys provide secure, password-less authentication.
- Enables secure file transfer between your local machine and remote servers.
- Essential for Linux administrators, DevOps and SRE engineers.

2. KEY COMMANDS

Command	What it does	Example
ssh	Connects to a remote server using SSH.	ssh user@192.168.1.10
scp	Securely copies files between local and remote systems.	scp file.txt user@server:/path
sftp	Interactive file transfer over SSH.	sftp user@192.168.1.10
ssh-keygen	Generates a new SSH key pair (public and private key).	ssh-keygen -t ed25519 -C "your_email@example.com"

3. UNDERSTANDING SSH



- SSH (Secure Shell) provides an encrypted connection between your machine and a remote server.
- You can authenticate using a password or, more securely, an SSH key pair.
- All data (commands and files) is encrypted.

4. GENERATE AN SSH KEY PAIR

```
$ ssh-keygen -t ed25519 -C "your_email@example.com"
```

Generating public/private ed25519 key pair.
Enter file in which to save the key (/Users/you/.ssh/id_ed25519):
Enter passphrase (optional):
Enter same passphrase again (optional):
Your identification has been saved in /Users/you/.ssh/id_ed25519.
Your public key has been saved in /Users/you/.ssh/id_ed25519.pub

This creates a private key (id_ed25519) and a public key (id_ed25519.pub).

5. CONNECT TO A REMOTE SERVER

```
$ ssh user@192.168.1.10
```

The authenticity of host '192.168.1.10' can't be established.
ED25519 key fingerprint is SHA256:0B6...
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.1.10' (ED25519) to the list of known hosts.
user@192.168.1.10's password:
[user@server ~]\$

The first time you connect, SSH will ask you to verify the server's identity (host key). This prevents man in the middle attacks.

6. TRANSFER FILES WITH scp AND sftp

Using scp (single command)

```
# Copy a file to remote server
$ scp file.txt user@server:/path

# Copy a file from remote server
$ scp user@server:/path/file.txt ./file.txt

# Copy a directory
$ scp -r myfolder/ user@server:/path
```

Using sftp (interactive)

```
$ sftp user@192.168.1.10
sftp> put file.txt
sftp> get file.txt
sftp> ls
sftp> cd /path
sftp> lpwd
sftp> bye
```

Both scp and sftp use SSH for secure file transfer.

7. SECURE KEY PERMISSIONS

```
# Private key permissions (must be 600)
$ chmod 600 ~/.ssh/id_ed25519

# Public key permissions (can be 644)
$ chmod 644 ~/.ssh/id_ed25519.pub

# .ssh directory permissions (must be 700)
$ chmod 700 ~/.ssh
```

If permissions are too open, SSH may refuse to use the key for security reasons.

9. IMPORTANT SECURITY NOTES

- Never commit private keys to Git repositories.
- Never share your private key with anyone.
- Use strong passphrases for additional security.
- Regularly rotate and remove unused keys.
- Be careful when accepting new host keys.

8. TROUBLESHOOTING: Permission denied (publickey)

```
$ ssh user@192.168.1.10
```

Permission denied (publickey).

COMMON CAUSES

- Public key not added to the server.
- Wrong user or hostname.
- Incorrect key permissions.
- Using the wrong private key.
- SSH service not running on server.
- Firewall blocking SSH (port 22).

HOW TO FIX

- Confirm you are using the correct private key (~/.ssh/to/key).
- Ensure the public key is added to ~/.ssh/authorized_keys on the server.
- Check file and directory permissions on the client and server.
- Verify the SSH service is running on the server (systemctl status sshd).
- Check firewall rules (port 22 must be open).
- Use ssh -v for detailed debug output.

10. JUNIOR ENGINEER TIPS

- Use SSH keys instead of passwords.
- Keep your private keys safe and never share them.
- Verify the server's host key when connecting for the first time.
- Use meaningful key comments (-C "your_email@example.com").
- Use ssh -v when troubleshooting connection issues.
- Consider using a dedicated key per environment (dev, staging, prod).



@veriqta



VERIQTA



VERIQTA



@veriqta_academy



VERIQTA



@veriqta

The Linux Administrator Troubleshooting Workflow

1. Application is unavailable

2. Check host state

3. Check service

4. Check process

5. Check logs

6. Check listening port

7. Check network / DNS

8. Check CPU, memory & disk

9. Identify root cause

10. Apply smallest safe fix

11. Verify recovery

1. APPLICATION IS UNAVAILABLE

- Users report they cannot access the application (e.g. website, API or internal service).
- Confirm the exact error (timeout, connection refused, 5xx, etc).

3. CHECK SERVICE

```
$ systemctl status <service>
$ systemctl is-active <service>
$ systemctl is-enabled <service>
```

Check if the service is running and enabled.

5. CHECK LOGS

```
$ journalctl -u <service> -n 50
$ journalctl -u <service> -f
$ journalctl -b
```

Look for recent errors, warnings and the real cause of the failure.

7. CHECK NETWORK / DNS

```
$ curl http://<domain-or-ip>
$ dig <domain>
$ getent hosts <domain>
$ traceroute <domain> # or tracepath
```

Check DNS resolution, reachability and network path to the service.

9. IDENTIFY ROOT CAUSE

- Use the evidence collected to identify the most likely cause, for example:
 - Service not running
 - Process crashed
 - Port not listening
 - DNS or network issue
 - High CPU or memory usage
 - Disk full
 - Application error in logs

11. VERIFY RECOVERY

- Confirm the application is working as expected.
- Test locally and from another server.
- Check logs for new errors.
- Monitor for a few minutes to ensure stability.

2. CHECK HOST STATE

```
$ uptime
$ hostname
$ hostnamectl
$ df -h
```

Check if the server is online, load is high and there is enough disk space.

4. CHECK PROCESS

```
$ ps aux | grep <process>
$ ps -ef | grep <process>
```

Verify the application process is running and check resource usage.

6. CHECK LISTENING PORT

```
$ ss -tulpn | grep <port>
$ ss -ltnp
$ curl http://localhost:<port>
```

Confirm the service is listening on the expected IP and port and test locally.

8. CHECK CPU, MEMORY & DISK

```
$ top
$ free -h
$ vmstat 1 5
$ df -h
$ du -sh /path/* | sort -h
```

Look for high CPU, memory pressure, swap usage or low disk space.

10. APPLY SMALLEST SAFE FIX

- Apply the smallest change needed. Examples:
 - Start the service: `systemctl start <service>`
 - Restart after confirming the cause: `systemctl restart <service>`
 - Fix configuration and reload: `systemctl reload <service>`
 - Free disk space or stop a runaway process
 - Fix DNS or network configuration

8. KEY LESSON

- Don't randomly restart things. Observe → gather evidence → form a hypothesis → test → fix → verify.

1. COMMON MISTAKES

- Restarting services without checking the cause.
- Ignoring logs and error messages.
- Not checking disk space, memory or CPU usage.
- Assuming the problem is the application when it is DNS or network.
- Making multiple changes at once.
- Not verifying the fix.

1. JUNIOR ENGINEER TIPS

- Always follow a logical troubleshooting flow.
- Use the right tool for each layer.
- Look for recent changes before making modifications.
- Document what you find and the fix.
- Monitor after the fix to ensure the problem does not return.